

FULL STACK

Name:Sadhana Nautiyal

Roll Number:45

Project Structure

```
labapp/
  templates/
    labapp/
      base.html
      home.html
      about.html
      contact.html
  views.py
  urls.py
  forms.py
```

Server log Output

After submitting the Contact Us form with test data, the console/server log shows:

```
INFO 2026-09-03 11:42:17,220 labapp Contact form submission | name=Sadhana Nautiyal |
email=sadhananautiyal@gmail.com | message=Requesting a callback regarding the internship
demo.
```

8.1 Active Navigation Highlight

Handled entirely in base.html above using request.resolver_match.url_name. No extra view logic is required because resolver_match is attached to every request automatically.

django.template.context_processors.request must be listed inside TEMPLATES > OPTIONS > context_processors in settings.py, or request will be undefined in templates and the whole {% if %} check will silently evaluate to false on every page.

```
# settings.py
TEMPLATES = [
    {
        ...
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    ],
]
```

8.2 Persistent Message Feedback Box

Django's built-in messages framework.

```
# settings.py
INSTALLED_APPS = [
    ...
    'django.contrib.messages',
]

MIDDLEWARE = [
    ...
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
]
```

MessageMiddleware depends on SessionMiddleware running first. If the order in MIDDLEWARE is swapped, or SessionMiddleware is missing, messages will raise a MessageFailure error instead of displaying.

Messages are consumed the moment the template renders them. Refreshing the same page a second time will not show the message again — this is expected behaviour, not a bug.

Home and About Us Pages

Both extend base.html and only override the content block.

```
{# home.html #}
{% extends "labapp/base.html" %}
{% block title %}Home{% endblock %}
{% block content %}
    <h1>Welcome Home</h1>
    <p>This is the home page.</p>
{% endblock %}

{# about.html #}
{% extends "labapp/base.html" %}
{% block title %}About Us{% endblock %}
{% block content %}
    <h1>About Us</h1>
    <p>Information about the organisation goes here.</p>
{% endblock %}
```

The path inside `{% extends "..."` must match the templates/<app>/ folder structure exactly, including case. "Labapp/base.html" and "labapp/base.html" are treated as different paths on case-sensitive filesystems (Linux servers), even if it works fine on Windows locally.

8.3 Contact Us Form with Server-Side Logging

A ModelForm-free simple form is used. On valid POST, the cleaned data is written to the server log using Python's logging module, then a success message is queued and the user is redirected (Post/Redirect/Get pattern, to prevent duplicate log entries on refresh).

```
# forms.py
from django import forms

class ContactForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()
    message = forms.CharField(widget=forms.Textarea)
```

```

# views.py
import logging
from django.shortcuts import render, redirect
from django.contrib import messages
from .forms import ContactForm

logger = logging.getLogger(__name__)

def home_view(request):
    return render(request, 'labapp/home.html')

def about_view(request):
    return render(request, 'labapp/about.html')

def contact_view(request):
    if request.method == 'POST':
        form = ContactForm(request.POST)
        if form.is_valid():
            name = form.cleaned_data['name']
            email = form.cleaned_data['email']
            message_text = form.cleaned_data['message']

            logger.info(
                "Contact form submission | name=%s | email=%s | message=%s",
                name, email, message_text
            )

            messages.success(request, "Thank you, your message has been received.")
            return redirect('labapp:contact')
        else:
            form = ContactForm()

    return render(request, 'labapp/contact.html', {'form': form})

# settings.py
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'handlers': {
        'console': {'class': 'logging.StreamHandler'},
    },
    'loggers': {
        'labapp': {
            'handlers': ['console'],
            'level': 'INFO',
        },
    },
}

{# contact.html #}
{% extends "labapp/base.html" %}
{% block title %}Contact Us{% endblock %}
{% block content %}

```

```

<h1>Contact Us</h1>
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Send</button>
</form>
{% endblock %}

```

{% csrf_token %} is mandatory inside every POST <form>. Missing it causes a 403 Forbidden on submit, which is a very common first-attempt error.

logger.info(...) only outputs if the LOGGING config's level is INFO or lower, and if the logger name in settings.py ('labapp') matches logging.getLogger(__name__) in views.py — __name__ resolves to 'labapp.views', so the settings.py logger entry must be the app name, not the full module path, for propagation to work.

Never log raw user input directly into a log line with string concatenation or f-strings if the message could contain newlines — use %s-style lazy formatting as shown above, so logging handles escaping safely instead of allowing log injection.

The redirect after a successful POST is required (Post/Redirect/Get). Skipping it and calling render() directly means refreshing the success page silently resubmits the form and logs the same message twice.

urls.py

```

from django.urls import path
from . import views

app_name = 'labapp'

urlpatterns = [
    path('', views.home_view, name='home'),
    path('about/', views.about_view, name='about'),
    path('contact/', views.contact_view, name='contact'),
]

```

Watch out: app_name = 'labapp' must be set for the {% url 'labapp:home' %} style tags used throughout base.html to resolve. Without it, Django raises NoReverseMatch.